

```

delayMicroseconds(LEDTime);

digitalWrite(GreenLEDPins[1/3],
LOW);
pinMode(GreenLEDPins[1/3],
INPUT);
pinMode(CathodePins[i3], IN-
PUT);
}
}
LEDOnOff = LEDOnOff >> 1;
LEDColour = LEDColour >> 1;
}
}

// readGrid function reads a press on
the wire grid
word readGrid()
{
int PinRowVal, PinColVal;
int SelectRow = 0;
int SelectCol = 0;

// set columns to OUTPUT LOW, set rows
to INPUT and enable pullups
for (int i=0;i<3;i++)
{
pinMode(i7+i, OUTPUT);
digitalWrite(i7+i, LOW);
pinMode(i4+i, INPUT);
digitalWrite(i4+i, HIGH);
}

// do an analog read of the row pins;
stop when the first one
// drops below the threshold
for (int RowCount=0;RowCount<3;RowC
ount++)
{
PinRowVal = 0;
for (int i=0;i<CountLoop;i++)
{
PinRowVal +=
analogRead(RowCount);
}
if (PinRowVal <
(CountLoop*Threshold))
{
SelectRow = RowCount + 1;
break;
}
}

// set rows to OUTPUT LOW, set columns
to INPUT and enable pullups
for (int i=0;i<3;i++)
{
pinMode(i4+i, OUTPUT);
digitalWrite(i4+i, LOW);
pinMode(i7+i, INPUT);
digitalWrite(i7+i, HIGH);
}

for (int ColCount=3;ColCount<5;Col
Count++)
{
PinColVal = 0;
for (int j=0;j<CountLoop;j++)
{
PinColVal +=
analogRead(ColCount);
}
if (PinColVal <
(CountLoop*Threshold))
{
SelectCol = ColCount-2;
break;
}
}
if ((SelectRow > 0) && (SelectCol >
0))
{
return (1 <<
(SelectRow*(SelectCol-1)*3)-1);
} else
{
return (0);
}
}

word checkWin(word GridOnOff, word Grid-
Colour, boolean Turn)
{
// there are 8 win conditions - three
column wins, three row wins, two diago-
nal wins
// this is put in an array, and the
GridOnOff status is just compared again
this
// function then returns the win condi-
tion
word winArray[] = {7, 56, 73, 84, 146,
273, 292, 448};

if (Turn) // red's turn, check
for green
{
for (int i=0;i<8;i++)
{
if ((GridOnOff & ~GridColour) &
winArray[i]) == winArray[i])
{
return winArray[i];
}
}
return 0;
} else // green's turn, check
for red
{
for (int i=0;i<8;i++)
{
if ((GridOnOff & GridColour)
& winArray[i]) == winArray[i])
{
return winArray[i];
}
}
return 0;
}

void displayWin(word winCondition,
boolean Turn)
{
word winColour;

if (Turn)
{
winColour = 0;
} else
{
winColour = 511;
}

for (int i=256;i>=0;i--)
{
lightLED(winCondition, winCol-
our); // light up the winning combo
lightLEDPWM(~winCondition, ~win-
Colour, i); // fade out the other
colour
}

for (int i=0;i<10;i++) // blink
winning combo a few times
{
lightLED(winCondition, winCol-
our);
delay(100);
}

void loop()
{
word Pressed;
word WinCondition = 0;
byte NoOfTurns = 0;
word GridOnOff = 0;
word GridColour = 0;
boolean Turn = 1; // 1 = red's
turn, 0 = green's turn
boolean Mode;
char RandomToss; // for play-
ing against uc
word PickedCell;

// turn on the LED on pin 13 depending
on the state of the ModePin
Mode = digitalRead(ModePin);
digitalWrite(ModeLEDPin, Mode);

// begin the game loop
// if one player, player starts first
do
{
Pressed = readGrid(); // take
a reading
if (Pressed & ~GridOnOff) // if
an empty space is selected
{
GridOnOff = GridOnOff | Pressed;
// light up the space
if (Turn)
// set colour according to whose turn
it is
{
GridColour = GridColour |
Pressed;
}

Turn = !Turn;
// Turn goes to opposite player
NoOfTurns++;
WinCondition =
checkWin(GridOnOff, GridColour, Turn);
// check if there's a win for player
if ((Mode) && !WinCondition))
// if uc's turn, play it now, unless
player has won
{
do // ran-
domly pick a blank cell
{
RandomToss = random(9);
PickedCell = 1;
for (int i=0;i<RandomToss;i++)
{
PickedCell = PickedCell <<
1;
} while ((PickedCell & Grid-
dOnOff));
GridOnOff = GridOnOff | Picked-
Cell;
Turn = !Turn;
// Turn goes to opposite player
NoOfTurns++;
WinCondition =
checkWin(GridOnOff, GridColour, Turn);
// check if there's a win for uc
}

lightLED(GridOnOff, GridColour);
// light up the LED
} while ((WinCondition == 0) && (NoOf-
Turns < 9));

if (WinCondition > 0) //
did anybody win?
{
displayWin(WinCondition, Turn);
} else //
it was a draw, fade out all lights
{
for (int i=512;i>=0;i--)
{
lightLEDPWM(GridOnOff, GridCol-
our, i/2);
}
}
}
}

```